Assignment 03 - Message Passing

Problem Analysis

The assignment focuses on concurrent programming based on message passing through two different models:

- asynchronous message passing with actors, implemented in Java with Apache Pekko typed;
- synchronous message passing with processes and channels, implemented in Go.

The common design constraint is to avoid shared mutable state between concurrent entities. Coordination is expressed through messages, and each component owns only its local state.

Exercise 1 - Smart Home Alarm System

Requirements

The smart home alarm control unit has five states:

- `DISARMED`: sensors are ignored;
- `EXIT_DELAY`: started after a valid arming PIN, sensors are still inactive;
- `ARMED`: active sensors can trigger an intrusion;
- `ENTRY_DELAY`: countdown after an active sensor event;
- `ALARM`: siren active until a valid PIN is entered.

The system supports configurable exit and entry delays. It also implements the optional zone-based extension: each sensor belongs to a zone and partial arming activates only selected zones.

Actor Architecture

The implementation is in package `pcd.ass03.alarm`.

```
AlarmSystemActor
  |-- ControlPanelActor
  |-- KeypadActor
  |-- SensorActor(front-door)
```

```
  |-- SensorActor(living-motion)
  |-- ...
```

`AlarmSystemActor` is the guardian/supervisor. It spawns the control panel, keypad, and sensors. External commands are sent to this actor and then forwarded to the appropriate child. The control panel is supervised with restart strategy.

`ControlPanelActor` is the state machine. Each alarm state is represented by a different Pekko `Behavior`, and transitions are performed by returning the next behavior from the message handler. This follows actor macro-step semantics: a message handler completes before the actor processes the next mailbox message, so internal state transitions do not require locks.

`KeypadActor` translates user keypad commands into control-panel commands. `SensorActor` translates simulated sensor triggers into `SensorTriggered` messages.

State Transitions

```
DISARMED -- valid PIN --> EXIT_DELAY -- timeout --> ARMED
ARMED -- active sensor --> ENTRY_DELAY -- timeout --> ALARM
ENTRY_DELAY -- valid PIN --> DISARMED
ALARM -- valid PIN --> DISARMED
```

Sensors in inactive zones are ignored while armed. Sensors are also ignored in `DISARMED`, `EXIT_DELAY`, `ENTRY_DELAY`, and `ALARM` according to the assignment specification.

Timers are implemented with `Behaviors.withTimers`, not with explicit sleep threads.

Running The Demo

```
.\mvnw.cmd exec:java
```

The demo performs partial arming on the `perimeter` zone, shows that an inactive `living` sensor is ignored, triggers the `front-door` sensor, and disarms during entry delay.

Observed output:

```
initial: StateSnapshot[state=DISARMED, activeZones=[], sirenOn=false]
after exit delay: StateSnapshot[state=ARMED, activeZones=[perimeter],
sirenOn=false]
inactive living sensor ignored: StateSnapshot[state=ARMED, activeZones=
```

```
[perimeter], sirenOn=false]
entry delay started: StateSnapshot[state=ENTRY_DELAY, activeZones=
[perimeter], sirenOn=false]
disarmed by keypad: StateSnapshot[state=DISARMED, activeZones=[],
sirenOn=false]
```

Exercise 2 - Odds-and-Evens Game

Requirements

The Go program implements an elimination tournament with `N = 2^m` players. Each round runs all matches concurrently. Winners advance to the next round, and the tournament ends when one player remains.

No shared memory is used for coordination. Players and matches communicate through Go channels.

Go Architecture

The implementation is in `go-odds-evens`.

Main elements:

- `playerAgent`: goroutine owning one player and receiving `PlayRequest` messages;
- `runMatch`: goroutine coordinating two players through channels and sending a `MatchResult`;
- `RunTournament`: coordinator that starts all matches for a round and performs fan-in on the results channel.

Each round is a barrier: the coordinator waits for all match results before creating the next round. Match results are stored by match index so the bracket order is deterministic even though matches complete concurrently.

The number of players is validated with a power-of-two check. The Go version also supports cancellation through a `done` channel and per-match timeout with `select`, following the CSP patterns discussed in the course. If cancellation happens, all goroutines observe the same `done` signal and stop without sharing mutable state.

Running The Go Program

```
cd go-odds-evens
```

```
go run . 8
go test -count=1 ./...
```

Verification

Java/Pekko tests:

```
.\mvnw.cmd test
```

Result:

```
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Packaging:

```
.\mvnw.cmd package
```

Result: `BUILD SUCCESS`.

Go formatting and tests:

```
gofmt -w main.go main_test.go
go test -count=1 ./...
go test -race -count=1 ./...
go run . 8
```

Result:

```
ok      pcd-ass03-odds-evens
ok      pcd-ass03-odds-evens    1.678s
Winner: P7 (id=7)
```